Digital Research Toolkit for Linguists

Week 5: Data manipulation

Anna Pryslopska

May 6, 2025

Psycholinguistics and Cognitive Modeling Lab

Homework Task 1

Clean up the Moses illusion data like we did in the tasks in class and save it to a new data frame.

```r
# - select relevant columns
select(moses, MD5.hash.of.participant.s.IP.address, ITEM, CONDITION, Value)

# - rename mislabeled columns
moses_cleaned <- rename(moses,
                ID = MD5.hash.of.participant.s.IP.address,
                ANSWER = Value)

# - remove missing data
na.omit(moses_cleaned$Item)

# - remove unnecessary rows
moses_cleaned <- na.omit(moses_cleaned)

# - arrange by condition, and answer
moses_cleaned <- arrange(moses_cleaned, CONDITION, ANSWER)
```

→ Unless you assign the values to a variable, they dissapear.

→ We didn't discuss pipes yet.

Homework Task 2

Why do these functions not work as intended? Fix the code and explain what was wrong.

```
read_csv(moses.csv)
tail(moses, n==10)
Summary(moses)
describe(Moses)
filter(moses, CONDITION == 102)
arragne(moses, ID)
```

✔ Typed the answer next to `# ANSWER`

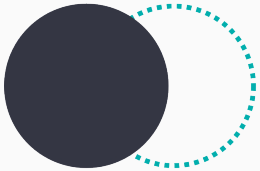
Homework Task 3

From the Moses illusion data, make two new variables (called ‘nobel’ and ‘valentines’, respectively) with all answers which are supposed to mean “Nobel Prize” and “Valentines Day”.

```
# Item nr. 16
nobel <- c("nobel", "the Nobel Prize", "Nobel Price",
"Nobel Prize", "Nobel awards", "Nobel prize",
"The Nobel Prize")

# Item nr. 109
valentines <- c("VALENTINE's DAY", "Valentine",
"Valentine's Day", "Valentine's day", "Valentines",
"Valentine's day ", "valentine's day")
```

Homework Task 4

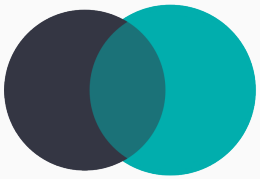


A

B

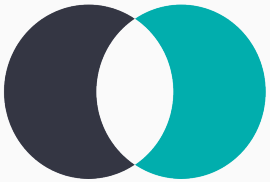
A & !B

!A & B



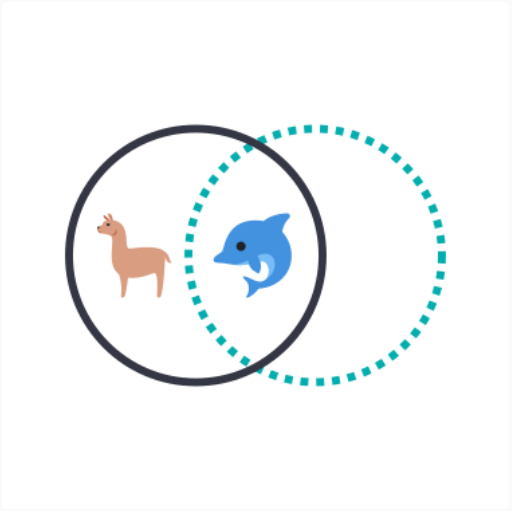
A | B

A & B

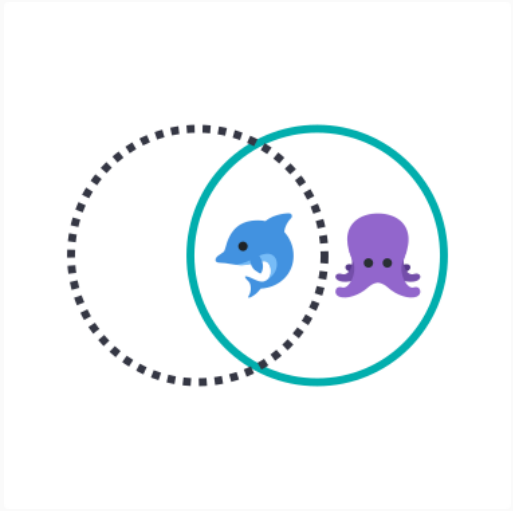


`xor(A, B)`

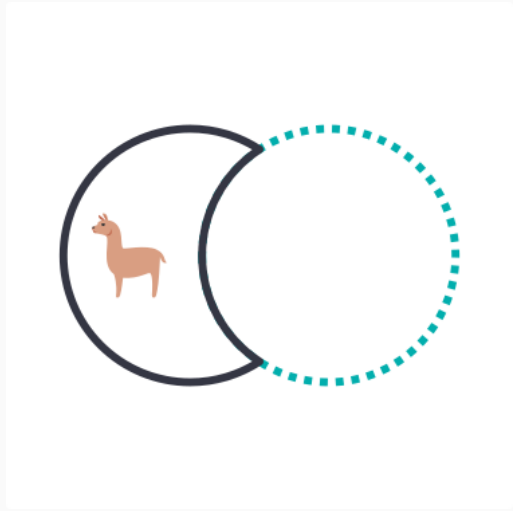
No `xor()` allowed!



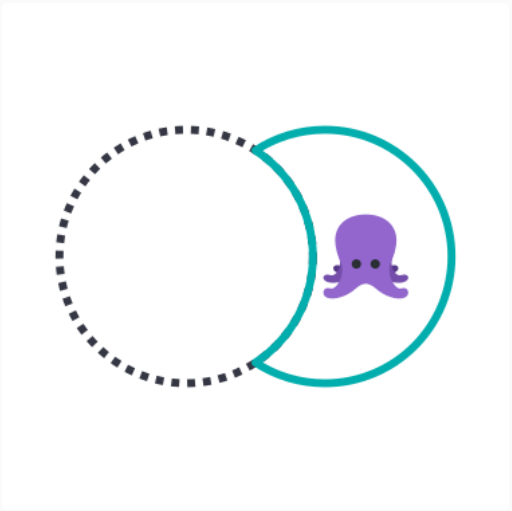
`mammal`



`dive`

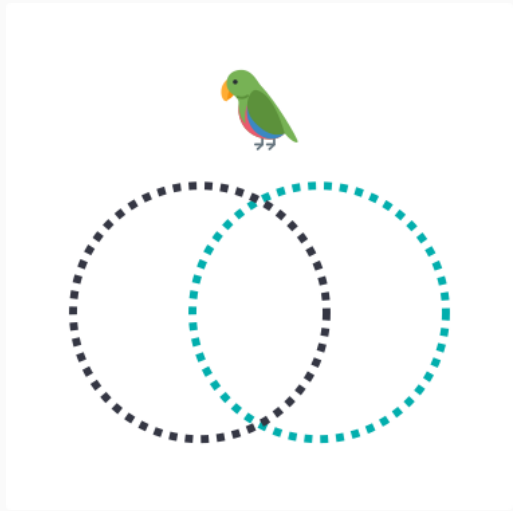


`mammal & !dive`

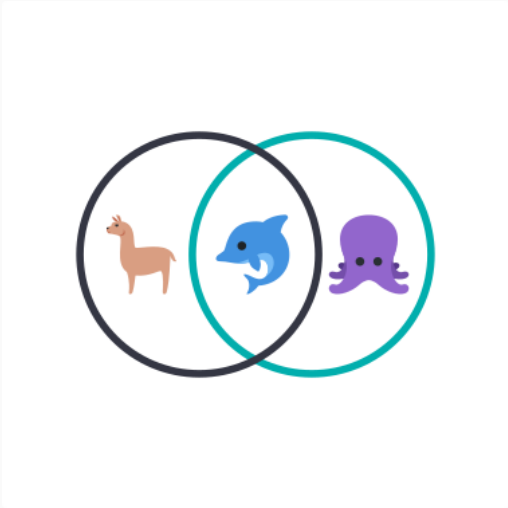


`!mammal & dive`

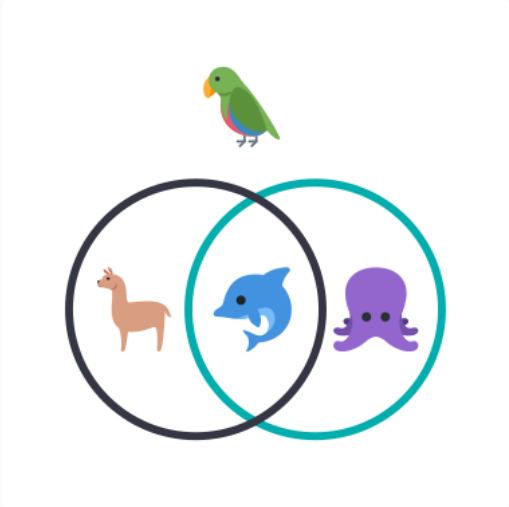
`mammal & dive`



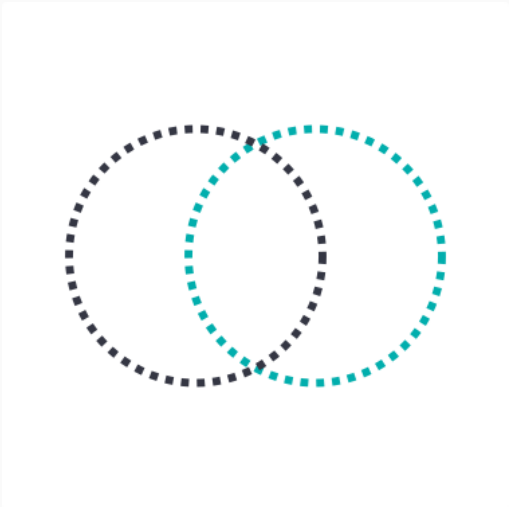
`!mammal & !dive`



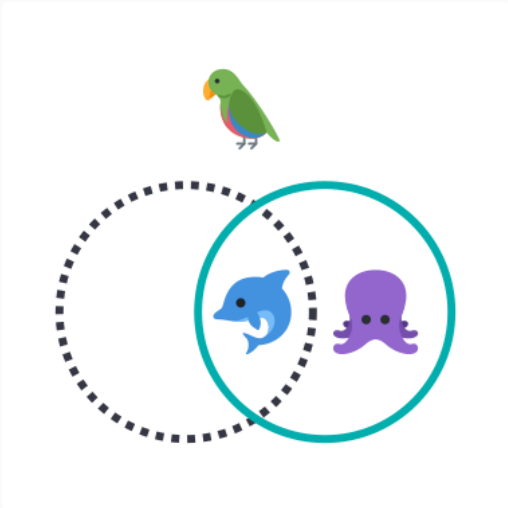
`mammal | dive`



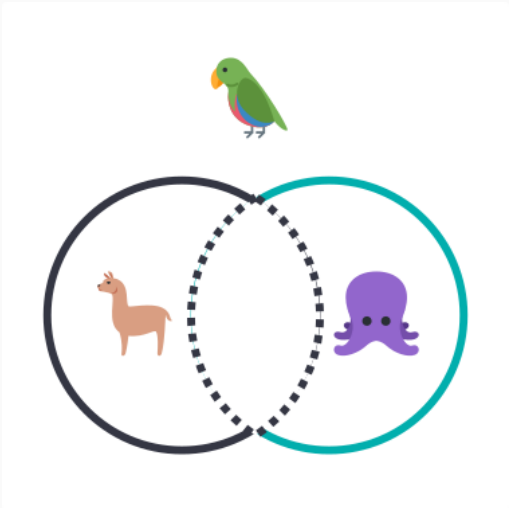
`mammal | !mammal`
`dive | !dive`



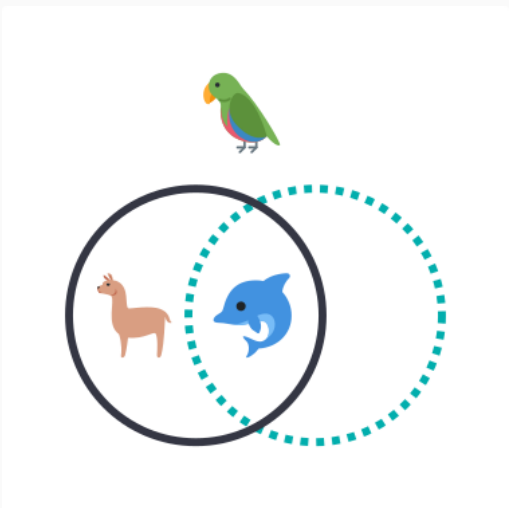
`mammal & !mammal`
`dive & !dive`



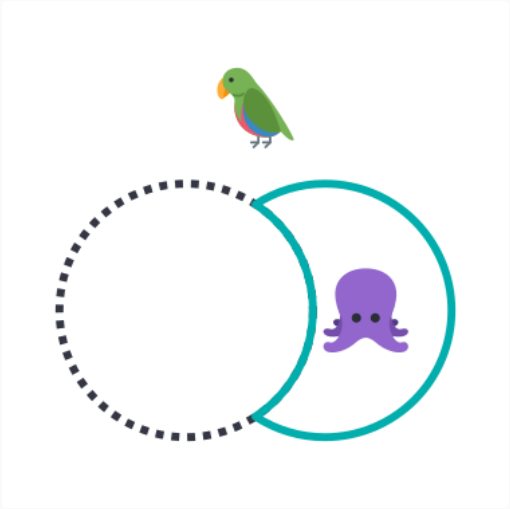
`!mammal | dive`
`!(mammal & !dive)`



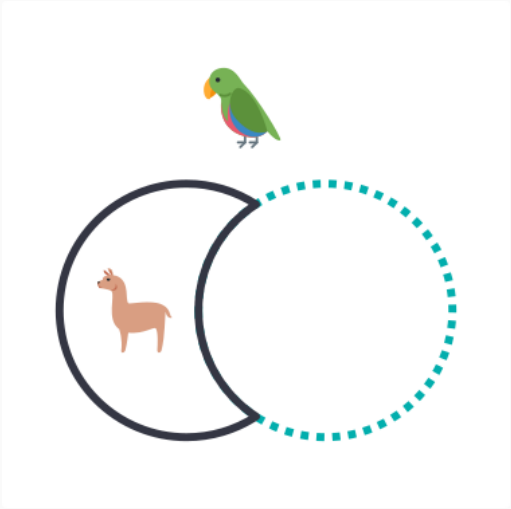
`!mammal | !dive`
`!(mammal & dive)`



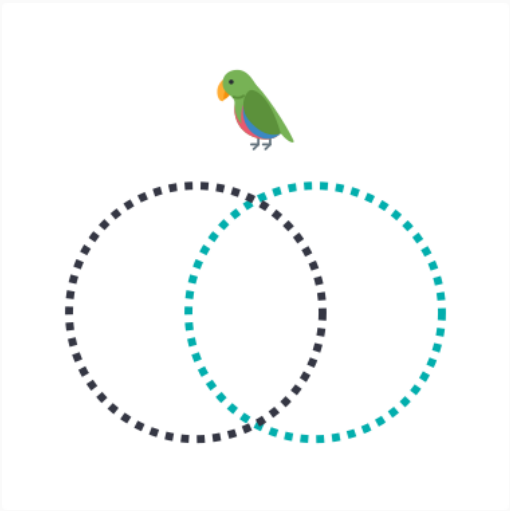
`?`



`!mammal`

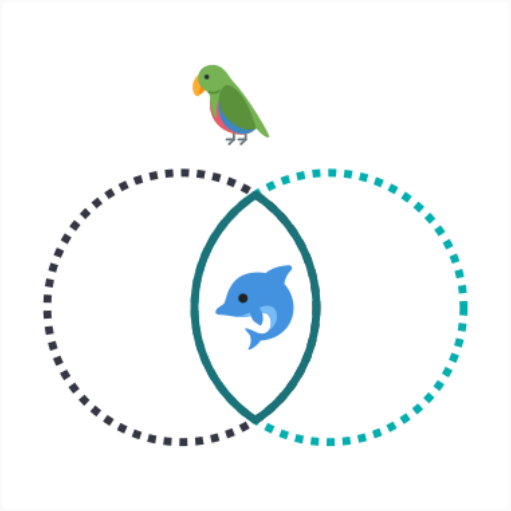


`!dive`



`!mammal & !dive`

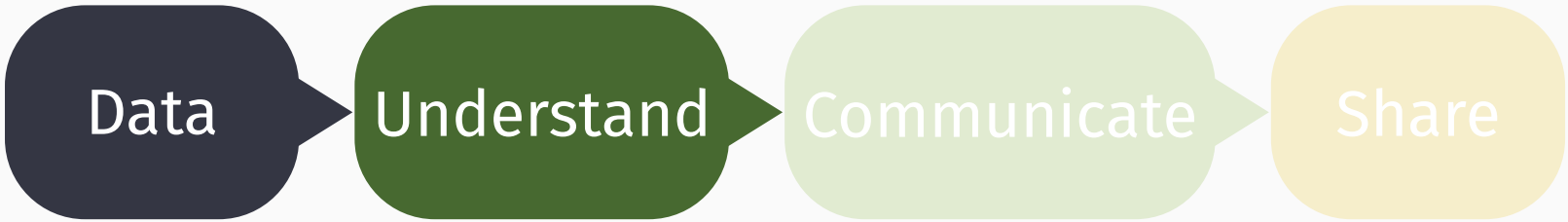
`mammal & dive`



( )|( )

Questions?

Table of contents



create, find, data types, formats, encoding

R, packages, import, clean, transform, debug, visualize, export

document, create clean and beautiful reports

connect, collaborate, backup, replicate

Mutating

Create and mutate

Tidyverse

```
mutate(moses, CLASS = TRUE)
```
make new column

```
mutate(moses, NUMBER = 1:11043)
```

```
mutate(moses, NUMBERS = NUMBER + 1)
```
calculate column

```
mutate(moses, NUMBER1 = NUMBER == 1)
```
evaluate column

```
mutate(moses, NUMBER = as.character(NUMBER))
```
overwrite column

```
mutate(moses, NUMBER1 = NULL)
```
remove column

Create and mutate

base R

```
moses$CLASS <- TRUE

moses$NUMBER <- 1:11043

moses$NUMBERS <- moses$NUMBER + 1

moses$NUMBER1 <- moses$NUMBER == 1

moses$NUMBER <- as.character(moses$NUMBER)

moses$NUMBER1 <- NULL
```

❗ Assignment saves, so be careful! This code deletes `NUMBER1` and permanently changes `NUMBER`.

Task 1: Re-code item number

Create a new data frame from the previous one, but with the `ITEM` column being numeric. Look at the first 20 rows of this new data frame.

```
mutate(WHAT, NEW = CHANGE HOW)
```

Pipes

Pipes

Pipes

A powerful tool for clearly expressing a sequence of multiple operations. Passes the output as the new input. They can be read as “and then”.

Created using `|>` (base) or `%>%` (`magrittr`). We will only use `|>`.

Why? It’s **native, newer, more readable, consistent** with other languages.

The pipe translates `x |> f(y)` into `f(x, y)`.

$\rightarrow$ “factoring out” `WHERE` ($\approx$ function application in lambda calculus)

Why bother?

Why? Simplify code, remove clutter and potential for error, reduce effort, stay reproducible.

Why not? No intermediate steps (need to run the whole code), writing functions is more complex.

When not? Very long pipes (>10 lines), multiple inputs or outputs, creating plots.

Data cleaning steps

give descriptive names	`rename( WHERE, NEW=OLD)`
select relevant columns	`select( WHERE, WHAT)`
remove missing values	`na.omit(WHERE)`
select values	`filter( WHERE, TRUE CONDITION)`
create values	`mutate( WHERE, NEW=FUNCTION(OLD))`
reorder values	`arrange(WHERE, HOW)`
check unique values	`unique( WHERE)`

Data cleaning with pipes

```
moses |>
  rename(ANSWER = Value,
         ID = MD5.hash.of.participant...) |>
  select(ID, ITEM, CONDITION, ANSWER) |>
  na.omit() |>
  filter(CONDITION != 0) |>
  mutate(ITEM = as.numeric(ITEM)) |>
  arrange(ITEM, CONDITION) |>
  unique()
```

Joining data frames

Add correct answers

Download preprocessed Moses illusion data and questions & answers from ILIAS and save them as `moses` and `questions`.
Put the two together using `merge()` (base R) or `*_join()` (`dplyr`)

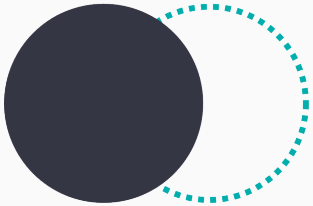
```
merge(x=DATA1, y=DATA2,
by=COLUMNS)
```

```
*_join(x=DATA1, y=DATA2,
by=COLUMNS)
```

Mutating joins

`left_join()`
keep all observations in `x`.
`right_join()`
keep all observations in `y`.
`full_join()`
keep all observations in `x` and `y`.
`inner_join()`
keep all observations from `x` that have a matching key in `y`

dpyr joins

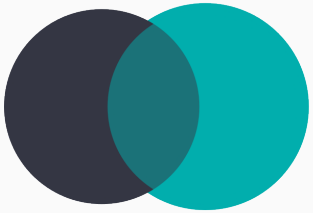


A
left_join(A,B)

B
right_join(A,B)

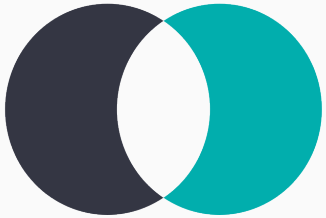
A & !B
anti_join(A,B)

!A & B
anti_join(B,A)



A | B
full_join(A,B)

A & B
inner_join(A,B)



xor(A, B)
it's complicated

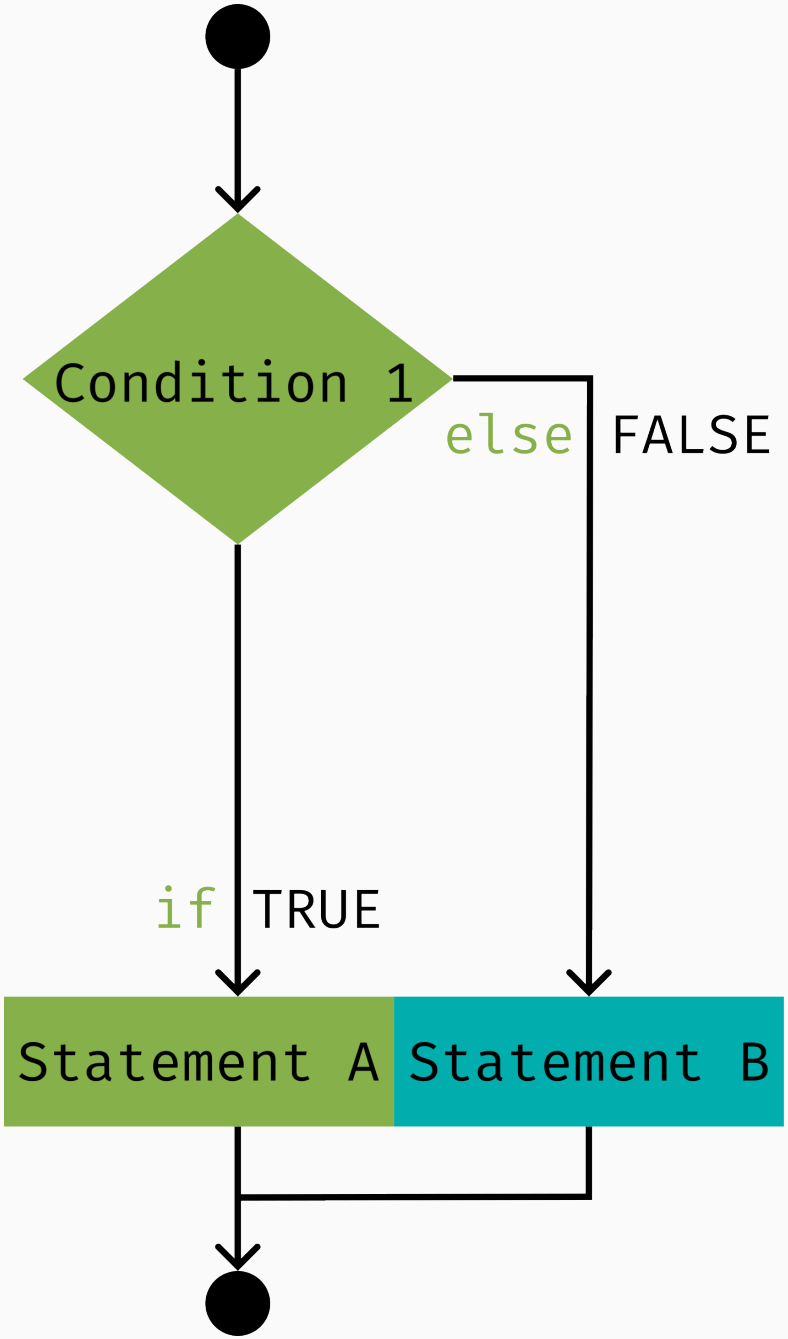
Task 2: Merge Moses illusion data with questions and answers

Using pipes, merge `moses` and `questions` by the columns `"ITEM", "CONDITION", "LIST"`. Then select the columns `ITEM, CONDITION, ID, ANSWER, CORRECT_ANSWER, LIST`.

```
merge(x = DATA1, y = DATA2, by = c(COLUMNS))
inner_join(x = DATA1, y = DATA2, by = c(COLUMNS))
full_join(x = DATA1, y = DATA2, by = c(COLUMNS))
```

If-else statements


DO IT OR ELSE!


Condition 1
else
FALSE
if
TRUE
Statement A
Statement B

Calculate accuracy: What if?

Was the answer correct or incorrect?

```
moses |>
  mutate(ACCURATE = CORRECT_ANSWER == ANSWER)          logical
```

```
ifelse(TEST, DO WHEN TRUE, DO WHEN FALSE)
```

```
moses |>
  mutate(ACCURATE = ifelse(test = CORRECT_ANSWER ==
ANSWER, yes = TRUE, no = FALSE))                       logical
```

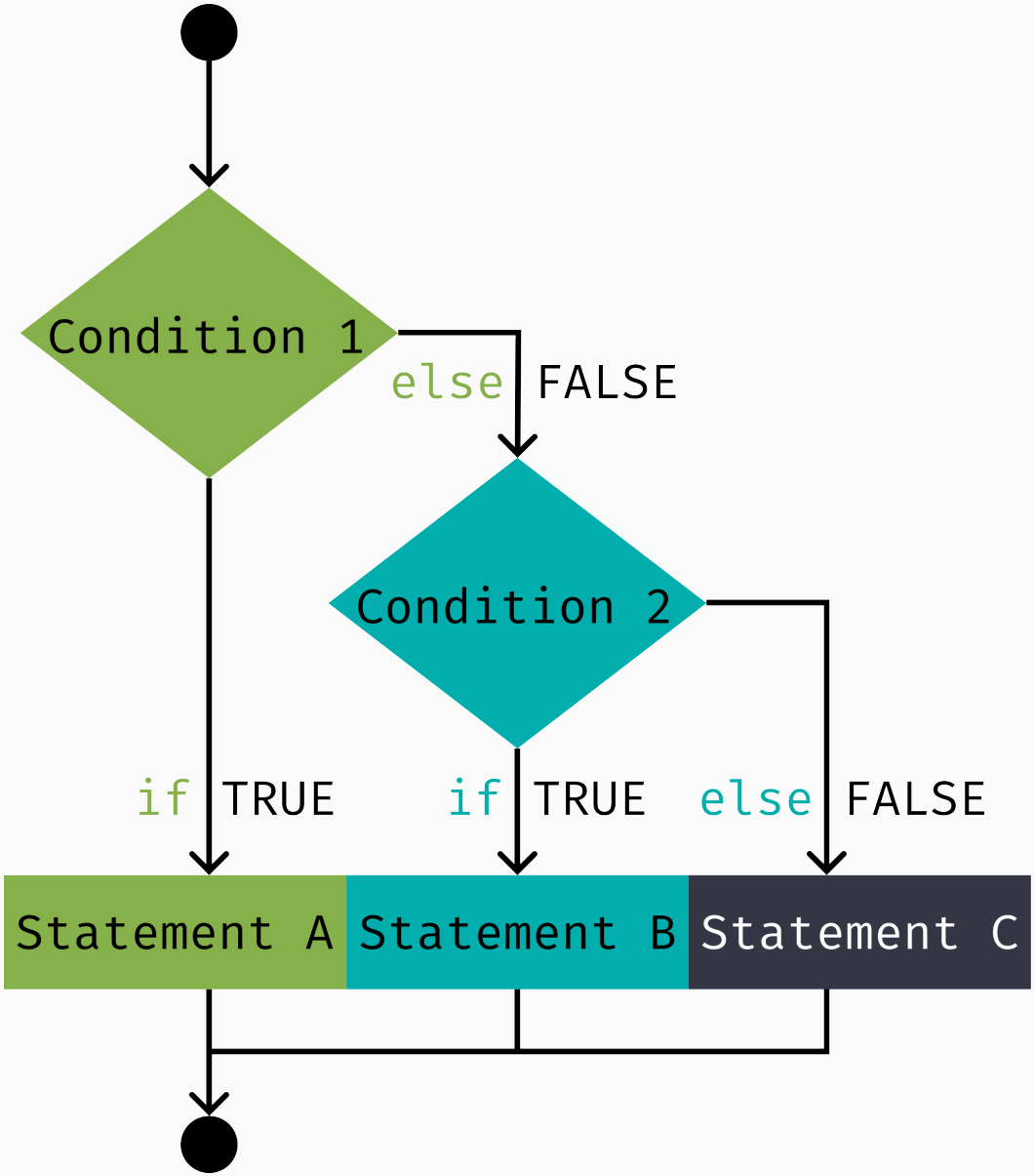
```
moses |>
  mutate(ACCURATE = ifelse(CORRECT_ANSWER == ANSWER,
"correct", "incorrect"))                               strings
```

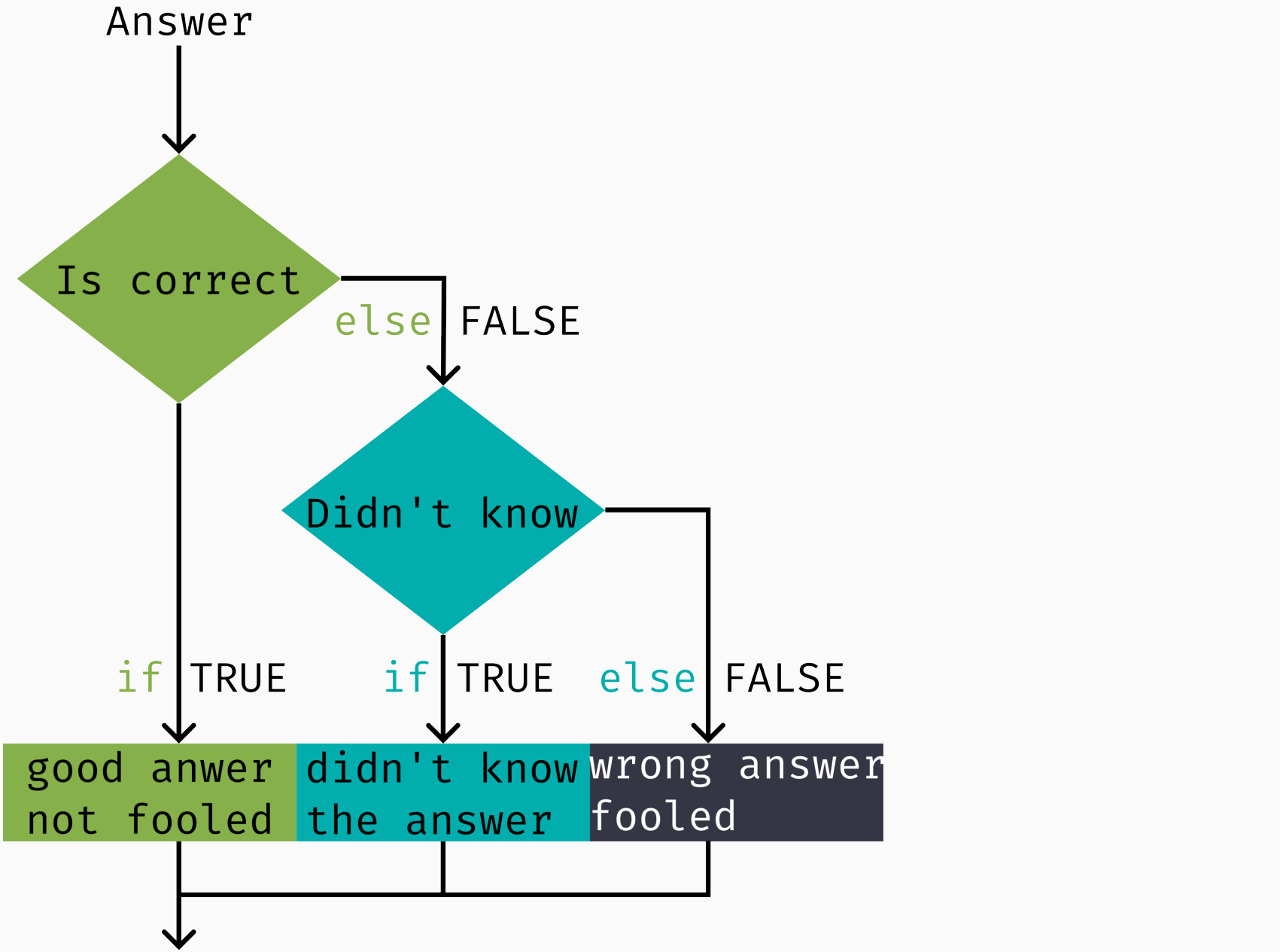
Task 3: Did you fall for the illusion?

Was the answer correct or incorrect? Make a new column which compares the correct answer with the answer given.

```
moses |>
    inner_join(questions, by=c("ITEM", "CONDITION",
              "LIST")) |>
    select(ITEM, CONDITION, ID, ANSWER, CORRECT_ANSWER,
           LIST) |>
    mutate(ACCURATE = ANSWER == CORRECT_ANSWER)
```

Accuracy: correct or incorrect or don't know?




Answer
Is correct
else FALSE
Didn't know
if TRUE
if TRUE
else FALSE
good anwer not fooled
didn't know the answer
wrong answer fooled
Move on to the next answer, calculate accuracy, ...

Task 4: Did you fall for the illusion or know the answer?

Was the answer correct or incorrect or "don't know"? Make a new column which compares the correct answer with the answer given.

```
moses |>
    inner_join(…) |>
    select(…) |>
    mutate(ACCURATE = ifelse(TRUE CONDITION,
           yes = …, no = ifelse(…)))
```

Task 4: Did you fall for the illusion or know the answer?

```
moses |>
    mutate(ACCURATE = ifelse(CORRECT_ANSWER ==
ANSWER,
       yes = "correct",
       no = ifelse(ANSWER == "dont_know",
                   yes = "dont_know",
                   no = "incorrect")))
```

Goal of experiment: Trick, no treat

Groups/conditions in experiment

1. **Moses illusion**
   "What is the name of the first man to walk on the *sun*?"
2. **Well-formed question**
   "What is the name of the first man to walk on the *moon*?"

100 Well-formed control "In which country is Florida located?"

101 Bad control "Which Nordic country are coconut trees native to?"

Predictions

1	No correct answers but you will try to answer anyway
2	Correct answer is predefined (e.g. Armstrong)
100	Correct answer is predefined (e.g. USA)
101	No correct answers and you will notice this

Condition numbers are meaningless!

```
mutate(WHERE, NEW=case_when(TRUE CONDITION ~ NEW VALUE))
```

```
moses |>
    mutate(CONDITION = case_when(
           CONDITION == '1' ~ 'illusion',
           CONDITION == '2' ~ 'no illusion',
           CONDITION == '100' ~ 'good filler',
           CONDITION == '101' ~ 'bad filler')
          )
```

Task 5: Calculate accuracy with `case_when()`

```
moses |>
    mutate(ACCURATE = ifelse(CORRECT_ANSWER ==
ANSWER,
        yes = "correct",
        no = ifelse(ANSWER == "dont_know",
                    yes = "dont_know",
                    no = "incorrect")))

mutate(WHERE, NEW=case_when(TRUE CONDITION ~ NEW
VALUE))

moses |>
    mutate(ACCURATE = case_when(
        ANSWER == CORRECT_ANSWER ~ "correct",
        ANSWER != "dont_know" ~ "incorrect",
        TRUE ~ ANSWER)
```

Grouping and summarizing

Grouping

`group_by(WHERE, BY WHAT)`
changes the unit of analysis from the complete dataset to particular groups.

`ungroup(WHERE)`
undos grouping.

Invisible but useful for summaries: How did the groups compare?

Summarizing

```
summarise(WHERE, NEW=FUNCTION(VALUE))
```
calculates values.

```
summarise(my.awesome.data,
  Count = n(),                    count cases
  Mean = mean(RT),                average reading time
  SD = sd(RT),                    how spread out is the data
  Min = min(Rating),              minimal value
  Sum = sum(Value))               sum of values
```

❗ `mutate()` changes an existing column or adds a new one.
`summarise()` calculates a single value (per group), drops cols

Task 6: Did you get got?

Summarize the results from the Moses illusion experiment. Count the answer types per condition (correct, incorrect, don't know).

```
group_by(WHERE, WHAT)
summarize(WHERE, NEW = FUNCTION(VALUE))
```

```
moses |>
    …
    group_by(CONDITION, ACCURATE) |>
    summarise(Count = n())
```

Wrap-up

Summary

- ✔ mutating
- ✔ power of names
- ✔ joining data frames
- ✔ if...else
- ✔ grouping
- ✔ summarizing
- ⏩ Tidy code, Getting help, data visualization intro (no laptop needed)

Homework assignment due May 12th 12:00

Submit 1 R script.

- Read `https://r.qcbs.ca/workshop03/pres-en/workshop03-pres-en.html`
- Complete assignment 5 ($\rightarrow$ ILIAS)
- Name your files with name and assignment number, e.g. `pryslopskaassignment05.R`

⚠ Using the `magrittr` pipe in your homework will result in »failing the assignment«.
If you wish to use the `magrittr` pipe in your homework, you need to submit a 5 A4 page explanation why you needed to do so. ⚠